Algoritmos de optimización - Seminario

Nombre y Apellidos:

- Alberto Paparelli
- Raúl Sánchez Mondéjar

Url: https://colab.research.google.com/drive/1VUm6MIDbOJ0DqVIF11Q8Li17J4xbKU-b

Problema:

> 2. Organizar los horarios de partidos de La Liga

Descripción del problema:

- Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.
- Los horarios disponibles se conocen a priori y son los siguientes:

| Viernes | 20 | |--------- |---------- | | Sábado | 12, 16, 18, 20 | | Domingo | 12, 16, 18, 20 | | Lunes | 20 |

- En primer lugar se clasifican los equipos en tres categorías según el numero de seguidores(que tiene relación directa con la audiencia). Hay 3 equipos en la categoría A, 11 equipos de categoría B y 6 equipos de categoría C.
- Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan y en horario de sábado a las 20h (el mejor en todos los casos).

| | Categoría A | Categoría B | Categoría C | |---------- |:----------: |:----------: |:----------: | | Categoría A | 2 Millones | 1.3 Millones | 1 Millon | | Categoría B | | 0.9 Millones | 0.75 Millones |

- Si el horario del partido no se realiza a las 20 horas del sábado se sabe que se reduce según los coeficientes de la siguiente tabla.
- Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes.

| | Viernes | Sábado | Domingo | Lunes | |----- |--------- |-------- |--------- |------- | | 12h | - | 0.55 | 0.45 | - | | 16h | - | 0.7 | 0.75 | - | | 18h | - | 0.8 | 0.85 | - | | 20h | 0.4 | 1 | 1 | 0.4 |

- Es posible la coincidencia de horarios pero en este caso la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla dependiendo del número de coincidencias:

| Coincidencias | -% | |:----------: |:---: | | 0 | 0% | | 1 | 25% | | 2 | 45% | | 3 | 60% | | 4 | 70% | | 5 | 75% | | 6 | 78% | | 7 | 80% | | 8 | 80% |

- Los cálculos asociados a una jornada de ejemplo se realizan según se muestra en la siguiente tabla:

| Partido | Categorías | Horario | Base (Millones) | Ponderación | Base * Ponderación | Corrección Coincidencia | |:----------: |:----------: |:----------: |:----------: |:----------: |:----------: |:----------: |:----------: | | Celta - Real Madrid | B-A | V20 | 1.3 | 0.4 | 0.52 | 0.52 | | Valencia - R. Sociedad | B-A | S12 | 1.3 | 0.55 | 0.72 | 0.72 | | Mallorca - Eibar | C-C | S16 | 0.47 | 0.7 | 0.33 | 0.33 | | Athletic - Barcelona | B-A | S18 | 1.3 | 0.8 | 1.04 | 1.04 | | Leganés - Osasuna | C-C | S20 | 0.47 | 1 | 0.47 | 0.47 | | Villarreal - Granada | **B-C** | **D16** | **0.75** | **0.75** | **0.56** | **0.42** *(1) | | Alavés - Levante | **B-B** | **D16** | **0.9** | **0.75** | **0.68** | **0.51** *(2) | | Espanyol - Sevilla | B-B | D18 | 0.9 | 0.85 | 0.77 | 0.77 | | Betis - Valladolid | B-C | D20 | 0.75 | 1 | 0.75 | 0.75 | | Atlético - Getafe | B-B | L20 | 0.9 | 0.4 | 0.36 | 0.36 | ||||||| TOTAL: 5.88 |

*(1) 0.42 = 0.56 x 0.75

*(2) 0.51 = 0.68 x 0.75

(*) La respuesta es obligatoria

---

* ¿Cuantas posibilidades hay sin tener en cuenta las restricciones?

Datos:

- 10 partidos por jornada (20 equipos / 2)
- 10 franjas horarias disponibles: V20, S12, S16, S18, S20, D12, D16, D18, D20, L20

Sin restricciones, cada partido puede asignarse a cualquiera de las 10 franjas horarias, y se permite que varios partidos coincidan en la misma franja.

Esto es un problema de ASIGNACIÓN CON REPETICIÓN: a cada uno de los n partidos le asignamos una de las k franjas horarias posibles.

Por el principio multiplicativo:

- Partido 1 -> 10 opciones
- Partido 2 -> 10 opciones
- ...
- Partido 10 -> 10 opciones

Total = k^n = 10^10

In [60]:
```
n_partidos = 10   # partidos por jornada
k_horarios = 10   # franjas horarias disponibles

# Total de asignaciones posibles (asignación con repetición)
total_posibilidades = k_horarios ** n_partidos

print(f"Partidos por jornada: {n_partidos}")
print(f"Franjas horarias disponibles: {k_horarios}")
print(f"Total de posibilidades (sin restricciones): {k_horarios}^{n_partidos} = {total_posibilidades:,}")
```

```
Partidos por jornada: 10
Franjas horarias disponibles: 10
Total de posibilidades (sin restricciones): 10^10 = 10,000,000,000
```

Respuesta:

Total de posibilidades (sin restricciones):

$$10^{10} = 10,000,000,000$$

¿Cuantas posibilidades hay teniendo en cuenta todas las restricciones.

Restricciones: * Obligatoriamente debe haber al menos 1 partido el viernes. * Obligatoriamente debe haber al menos 1 partido el lunes.

Para contar las asignaciones válidas usamos el principio de inclusión-exclusión:

Sea:

- T = total sin restricciones = 10^{10}
- A = asignaciones donde **ningún** partido se juega el viernes → cada partido elige entre 9 slots = 9^{10}
- B = asignaciones donde **ningún** partido se juega el lunes → 9^{10}
- $A \cap B$ = asignaciones sin viernes **ni** lunes → cada partido elige entre 8 slots = 8^{10}

Aplicando inclusión-exclusión:

$$\text{Válidas} = T - |A| - |B| + |A \cap B| = 10^{10} - 2 \times 9^{10} + 8^{10}$$

In [61]:
```python
total = 10**10            # sin restricciones
sin_viernes = 9**10       # ningún partido el viernes (9 slots disponibles)
sin_lunes = 9**10         # ningún partido el lunes (9 slots disponibles)
sin_ambos = 8**10         # ningún partido el viernes ni el lunes (8 slots)

# Inclusión-exclusión: al menos 1 viernes Y al menos 1 lunes
validas = total - sin_viernes - sin_lunes + sin_ambos

print(f"Total sin restricciones:          {total:>15,}")
print(f"Sin viernes (9^10):               {sin_viernes:>15,}")
print(f"Sin lunes (9^10):                 {sin_lunes:>15,}")
print(f"Sin viernes ni lunes (8^10):      {sin_ambos:>15,}")
print(f"Total con restricciones:          {validas:>15,}")
```

```
Total sin restricciones:          10,000,000,000
Sin viernes (9^10):                3,486,784,401
Sin lunes (9^10):                  3,486,784,401
Sin viernes ni lunes (8^10):       1,073,741,824
Total con restricciones:           4,100,173,022
```

Respuesta:

Teniendo en cuenta la restricción de al menos 1 partido el viernes y 1 el lunes, el número de asignaciones válidas es:

$$10^{10} - 2 \times 9^{10} + 8^{10} = 4,100,173,022$$

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.
(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, arguementalo)

Respuesta: Vector (lista Python)

Representación:

Utilizamos una **lista de 10 enteros** donde la posición `i` representa el partido `i` y el valor almacenado indica el índice de la franja horaria asignada (0-9).

```
solucion = [3, 0, 5, 7, 4, 6, 6, 8, 9, 1]
              ^  ^  ^  ...
          P0->S18  P1->V20  P2->D12 ...
```

Mapeo de franjas horarias:

Índice	Franja
0	V20
1	S12
2	S16
3	S18
4	S20
5	D12
6	D16
7	D18
8	D20
9	L20

Argumentación:

1. **Consistencia con lo visto en clase:** Es la misma estructura utilizada en el problema de N-Reinas (AG1): un vector donde la posición representa la entidad a asignar y el valor representa la asignación. Además, el manual y las actividades guiadas usan vectores para problemas de asignación.

2. **Eficiencia computacional:** Acceso O(1) por índice, tamaño fijo conocido (n=10), fácil generación de vecinos (cambiar un valor en una posición), y compatible con fuerza bruta (recorrer las 10^{10} combinaciones mediante conteo en base 10).

3. **Verificación de restricciones simple:** Contar coincidencias por franja con un conteo manual mediante un vector auxiliar de tamaño k (O(n)), verificar la presencia de viernes (valor 0) o lunes (valor 9) con una simple comparación de valores en el vector.

Alternativa descartada, Diccionario `{franja: [partidos]}`:

La perspectiva inversa (agrupar partidos por franja) facilita calcular coincidencias, pero complica la generación de soluciones y la iteración del espacio de búsqueda. Esta vista se puede derivar fácilmente del vector cuando se necesite, por lo que no justifica usarla como estructura principal.

In [62]:
```python
# ----------------------------------------------------------------------------
# Definición de la representación
# ----------------------------------------------------------------------------

N_PARTIDOS = 10   # partidos por jornada
N_FRANJAS  = 10   # franjas horarias disponibles (índices 0-9)

# Mapeo índice → etiqueta legible de franja horaria
IDX_A_FRANJA = {
    0: "V20", 1: "S12", 2: "S16", 3: "S18", 4: "S20",
    5: "D12", 6: "D16", 7: "D18", 8: "D20", 9: "L20",
}

# Índices especiales para restricciones
IDX_VIERNES = 0   # V20
IDX_LUNES   = 9   # L20

# Ejemplo: jornada del enunciado
# P0=Celta-RMadrid, P1=Valencia-RSociedad, P2=Mallorca-Eibar,
# P3=Athletic-Barcelona, P4=Leganés-Osasuna, P5=Villarreal-Granada,
# P6=Alavés-Levante, P7=Espanyol-Sevilla, P8=Betis-Valladolid,
# P9=Atlético-Getafe
partidos = [
    "Celta-RMadrid", "Valencia-RSociedad", "Mallorca-Eibar",
    "Athletic-Barcelona", "Leganés-Osasuna", "Villarreal-Granada",
    "Alavés-Levante", "Espanyol-Sevilla", "Betis-Valladolid",
    "Atlético-Getafe",
]

# Solución de ejemplo (la del enunciado)
# V20, S12, S16, S18, S20, D16, D16, D18, D20, L20
solucion_ejemplo = [0, 1, 2, 3, 4, 6, 6, 7, 8, 9]

# ----------------------------------------------------------------------------
# Mostrar la asignación
# ----------------------------------------------------------------------------
print("Solución ejemplo (vector):", solucion_ejemplo)
print()
for i, franja_idx in enumerate(solucion_ejemplo):
    print(f"  P{i} ({partidos[i]:>22s}) -> franja {franja_idx} ({IDX_A_FRANJA[franja_idx]})")

# ----------------------------------------------------------------------------
# Verificación de restricciones
# ----------------------------------------------------------------------------
tiene_viernes = IDX_VIERNES in solucion_ejemplo
tiene_lunes = IDX_LUNES   in solucion_ejemplo
es_valida = tiene_viernes and tiene_lunes

print(f"\n¿Al menos 1 partido en viernes? {tiene_viernes}")
print(f"¿Al menos 1 partido en lunes?   {tiene_lunes}")
print(f"¿Solución válida?               {es_valida}")

# ----------------------------------------------------------------------------
# Coincidencias por franja (útil para penalizaciones)
# Conteo manual con vector auxiliar de tamaño k, O(n)
# ----------------------------------------------------------------------------
conteo = [0] * N_FRANJAS
for franja_idx in solucion_ejemplo:
    conteo[franja_idx] += 1

print("\nCoincidencias por franja:")
for franja_idx, cantidad in enumerate(conteo):
    if cantidad > 0:
        extra = f"  <- {cantidad - 1} coincidencia(s)" if cantidad > 1 else ""
        print(f"  {IDX_A_FRANJA[franja_idx]}: {cantidad} partido(s){extra}")
```

```
Solución ejemplo (vector): [0, 1, 2, 3, 4, 6, 6, 7, 8, 9]

  P0 (         Celta-RMadrid) -> franja 0 (V20)
  P1 (    Valencia-RSociedad) -> franja 1 (S12)
  P2 (        Mallorca-Eibar) -> franja 2 (S16)
  P3 (    Athletic-Barcelona) -> franja 3 (S18)
  P4 (       Leganés-Osasuna) -> franja 4 (S20)
  P5 (    Villarreal-Granada) -> franja 6 (D16)
  P6 (        Alavés-Levante) -> franja 6 (D16)
  P7 (      Espanyol-Sevilla) -> franja 7 (D18)
  P8 (      Betis-Valladolid) -> franja 8 (D20)
  P9 (       Atlético-Getafe) -> franja 9 (L20)

¿Al menos 1 partido en viernes? True
¿Al menos 1 partido en lunes?   True
¿Solución válida?               True

Coincidencias por franja:
  V20: 1 partido(s)
  S12: 1 partido(s)
  S16: 1 partido(s)
  S18: 1 partido(s)
  S20: 1 partido(s)
  D16: 2 partido(s)  <- 1 coincidencia(s)
  D18: 1 partido(s)
  D20: 1 partido(s)
  L20: 1 partido(s)
```

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

Función objetivo:

La función objetivo calcula la **audiencia total** (en millones de espectadores) de una jornada, sumando la contribución de cada partido. La contribución de cada partido depende de tres factores multiplicativos:

$$f(S) = \sum_{i=0}^{n-1} \text{base}_i \times \text{pond}(S_i) \times \text{penalización}(\text{coinc}_i)$$

donde, para cada partido i:

- base_i: audiencia base según las categorías de los dos equipos (en millones, referida al horario S20).
- $\text{pond}(S_i)$: coeficiente de la franja horaria asignada al partido i (entre 0.4 y 1.0).
- $\text{coinc}_i = (\text{partidos en franja } S_i) - 1$: número de coincidencias en esa franja.
- $\text{penalización}(c)$: factor de reducción según la tabla de coincidencias (0 coinc → 1.0, 1 coinc → 0.75, ...).

Es un problema de maximización. Queremos encontrar la asignación de partidos a franjas horarias que maximice la audiencia total de la jornada.

Complejidad de evaluación: O(n) para evaluar una solución (un recorrido para contar coincidencias por franja + un recorrido para sumar audiencias).

In [63]:
```python
# --- Datos del problema ---
# Audiencia base (millones) según categorías, simétrica
BASE_AUDIENCIA = {
    ("A","A"): 2.0,  ("A","B"): 1.3,  ("A","C"): 1.0,
    ("B","A"): 1.3,  ("B","B"): 0.9,  ("B","C"): 0.75,
    ("C","A"): 1.0,  ("C","B"): 0.75, ("C","C"): 0.47,
}

# Ponderación por franja horaria (índice 0-9)
PONDERACION = [0.4, 0.55, 0.7, 0.8, 1.0, 0.45, 0.75, 0.85, 1.0, 0.4]

# Factor de penalización por número de coincidencias en la misma franja
PENALIZACION = [1.0, 0.75, 0.55, 0.40, 0.30, 0.25, 0.22, 0.20, 0.20]

# Categorías de los equipos del ejemplo
categorias_ejemplo = [
    ("B","A"), ("B","A"), ("C","C"), ("B","A"), ("C","C"),
    ("B","C"), ("B","B"), ("B","B"), ("B","C"), ("B","B"),
]

def funcion_objetivo(solucion, categorias):
    """Calcula la audiencia total (en millones) de una asignación."""
    n = len(solucion)

    # Paso 1: contar partidos por franja, O(n)
    conteo = [0] * N_FRANJAS
    for franja in solucion:
        conteo[franja] += 1

    # Paso 2: sumar audiencia ponderada y penalizada, O(n)
    audiencia_total = 0.0
    for i in range(n):
        base  = BASE_AUDIENCIA[categorias[i]]
        pond  = PONDERACION[solucion[i]]
        coinc = conteo[solucion[i]] - 1          # coincidencias = partidos_en_franja - 1
        penal = PENALIZACION[coinc]
        audiencia_total += base * pond * penal

    return audiencia_total

# --- Validación con el ejemplo del enunciado ---
total = funcion_objetivo(solucion_ejemplo, categorias_ejemplo)
print(f"Audiencia total (ejemplo): {total:.4f} millones")
print(f"Valor esperado (enunciado, redondeado): 5.88 millones")

# Desglose por partido
print("\nDesglose:")
conteo = [0] * N_FRANJAS
for f in solucion_ejemplo:
    conteo[f] += 1

for i in range(N_PARTIDOS):
    base  = BASE_AUDIENCIA[categorias_ejemplo[i]]
    pond  = PONDERACION[solucion_ejemplo[i]]
    coinc = conteo[solucion_ejemplo[i]] - 1
    penal = PENALIZACION[coinc]
    valor = base * pond * penal
    print(f"  P{i} {partidos[i]:>22s}  base={base:.2f}  pond={pond:.2f}"
          f"  coinc={coinc}  penal={penal:.2f}  -> {valor:.4f}")
```

```
Audiencia total (ejemplo): 5.8771 millones
Valor esperado (enunciado, redondeado): 5.88 millones

Desglose:
  P0          Celta-RMadrid  base=1.30  pond=0.40  coinc=0  penal=1.00  -> 0.5200
  P1     Valencia-RSociedad  base=1.30  pond=0.55  coinc=0  penal=1.00  -> 0.7150
  P2         Mallorca-Eibar  base=0.47  pond=0.70  coinc=0  penal=1.00  -> 0.3290
  P3     Athletic-Barcelona  base=1.30  pond=0.80  coinc=0  penal=1.00  -> 1.0400
  P4        Leganés-Osasuna  base=0.47  pond=1.00  coinc=0  penal=1.00  -> 0.4700
  P5     Villarreal-Granada  base=0.75  pond=0.75  coinc=1  penal=0.75  -> 0.4219
  P6         Alavés-Levante  base=0.90  pond=0.75  coinc=1  penal=0.75  -> 0.5063
  P7       Espanyol-Sevilla  base=0.90  pond=0.85  coinc=0  penal=1.00  -> 0.7650
  P8       Betis-Valladolid  base=0.75  pond=1.00  coinc=0  penal=1.00  -> 0.7500
  P9        Atlético-Getafe  base=0.90  pond=0.40  coinc=0  penal=1.00  -> 0.3600
```

Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

Enfoque: Enumerar **todas** las k^n asignaciones posibles (donde $k = 10$ franjas y $n = 10$ partidos), verificar las restricciones para cada una, evaluar la función objetivo y quedarse con la asignación que maximice la audiencia total.

Pseudocódigo:

```
función FUERZA_BRUTA(categorías, n_partidos, n_franjas):
    mejor_solución <- nula
    mejor_audiencia  <- -∞
    solución ← [0, 0, ..., 0]          // primera combinación (n_partidos ceros)

    REPETIR:
        SI es_válida(solución):         // ¿tiene al menos 1 viernes y 1 lunes?
            audiencia <-> función_objetivo(solución, categorías)
            SI audiencia > mejor_audiencia:
                mejor_audiencia <- audiencia
                mejor_solución <- copia(solución)

        SI NO siguiente(solución, n_franjas):   // incrementar "odómetro" en base k
            SALIR                               // se recorrieron todas las combinaciones

    DEVOLVER mejor_solución, mejor_audiencia
```

Generación de combinaciones - conteo en base k (odómetro):

El vector solución se trata como un número en base k. La función `siguiente()` incrementa la última posición y propaga el acarreo hacia la izquierda, como un odómetro:

```
[0,0,0] -> [0,0,1] -> ... → [0,0,9] -> [0,1,0] → ... → [9,9,9] -> desbordamiento (fin)
```

Esto genera las k^n combinaciones sin necesidad de `itertools` ni recursión.

Inviabilidad para $n = 10$:

$10^{10} = 10{,}000{,}000{,}000$ combinaciones. Incluso evaluando 10^6 por segundo, se necesitarían $\approx 10{,}000$ segundos (≈ 2.8 horas). Es computacionalmente inviable.

Demostración con ejemplo reducido:

Para verificar la corrección del algoritmo, lo ejecutamos con los primeros 6 partidos de la jornada ejemplo: $10^6 = 1{,}000{,}000$ combinaciones, factible en pocos segundos.

In [64]:
```python
import time

def es_valida(solucion):
    """Restricciones: al menos 1 viernes (idx 0) y 1 lunes (idx 9)."""
    return IDX_VIERNES in solucion and IDX_LUNES in solucion

def siguiente(solucion, base):
    """Incrementa el vector en base k (odómetro). Devuelve False si desborda."""
    i = len(solucion) - 1
    while i >= 0:
        solucion[i] += 1
        if solucion[i] < base:
            return True
        solucion[i] = 0
        i -= 1
    return False  # desbordamiento → se recorrieron todas

def fuerza_bruta(categorias, n_partidos, n_franjas):
    """Enumera las k^n asignaciones por conteo en base k."""
    mejor_solucion = None
    mejor_audiencia = -1
    evaluadas = 0

    solucion = [0] * n_partidos      # primera combinación: [0,0,...,0]
    while True:
        if es_valida(solucion):
            evaluadas += 1
            audiencia = funcion_objetivo(solucion, categorias)
            if audiencia > mejor_audiencia:
                mejor_audiencia = audiencia
                mejor_solucion = solucion[:]
        if not siguiente(solucion, n_franjas):
            break

    return mejor_solucion, mejor_audiencia, evaluadas

# --- Demo con ejemplo reducido (6 partidos) ---
n_reducido = 6
cats_reducido = categorias_ejemplo[:n_reducido]

print(f"Fuerza bruta con {n_reducido} partidos "
      f"({N_FRANJAS}^{n_reducido} = {N_FRANJAS**n_reducido:,} combinaciones)")

t0 = time.time()
sol, aud, ev = fuerza_bruta(cats_reducido, n_reducido, N_FRANJAS)
t1 = time.time()

print(f"Soluciones válidas evaluadas: {ev:,}")
print(f"Tiempo: {t1-t0:.2f} s")
print(f"\nMejor audiencia: {aud:.4f} millones")
print(f"Mejor solución:  {sol}")
for i in range(n_reducido):
    print(f"  P{i} ({partidos[i]:>22s}) -> {IDX_A_FRANJA[sol[i]]}")

# --- Estimación para n=10 ---
factor = N_FRANJAS ** (N_PARTIDOS - n_reducido)
t_estimado = (t1 - t0) * factor
print(f"\nPara n={N_PARTIDOS}: {N_FRANJAS}^{N_PARTIDOS} = "
      f"{N_FRANJAS**N_PARTIDOS:,} combinaciones")
print(f"Tiempo estimado: {t_estimado:.0f} s "
      f"({t_estimado/3600:.1f} h) -> INVIABLE")
```

```
Fuerza bruta con 6 partidos (10^6 = 1,000,000 combinaciones)
Soluciones válidas evaluadas: 199,262
Tiempo: 0.33 s

Mejor audiencia: 4.6810 millones
Mejor solución:  [4, 7, 0, 8, 9, 3]
  P0 (         Celta-RMadrid) -> S20
  P1 (    Valencia-RSociedad) -> D18
  P2 (        Mallorca-Eibar) -> V20
  P3 (    Athletic-Barcelona) -> D20
  P4 (       Leganés-Osasuna) -> L20
  P5 (    Villarreal-Granada) -> S18

Para n=10: 10^10 = 10,000,000,000 combinaciones
Tiempo estimado: 3294 s (0.9 h) -> INVIABLE
```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

El algoritmo de fuerza bruta tiene tres operaciones principales que se ejecutan para cada una de las k^n combinaciones:

Operación	Coste por llamada	Llamadas
`siguiente()` : incrementar odómetro	$O(n)$ peor caso, $O(1)$ amortizado	k^n
`es_valida()` : buscar viernes y lunes en el vector	$O(n)$	k^n
`funcion_objetivo()` : contar coincidencias + sumar audiencias	$O(n)$	$\leq k^n$ (solo válidas)

Complejidad temporal:

Cada iteración del bucle realiza trabajo $O(n)$ (validación + evaluación + avance del odómetro), y hay k^n iteraciones:

$$T(n,k) = O(n \cdot k^n)$$

Con los valores del problema ($k = n = 10$):

$$T(10,10) = O(10 \cdot 10^{10}) = O(10^{11})$$

La complejidad es **exponencial** en n: El factor dominante es k^n, que crece mucho más rápido que el factor lineal n.

Complejidad espacial:

$$S(n,k) = O(n+k)$$

- $O(n)$: el vector solución actual + la mejor solución guardada.
- $O(k)$: el vector auxiliar de conteo por franja dentro de `funcion_objetivo`.
- No se almacenan todas las combinaciones en memoria; se generan una a una (generación implícita).

Clase de complejidad:

El algoritmo pertenece a la clase $O(k^n)$ tiempo exponencial. Es intratable para valores moderados de n y k, lo que motiva la búsqueda de algoritmos más eficientes (greedy, backtracking, metaheurísticas).

In [65]:
```python
# --- Complejidad del algoritmo de fuerza bruta ---
# T(n,k) = O(n · k^n)   |   S(n,k) = O(n + k)

print("Crecimiento de la complejidad temporal T(n,k) = n · k^n")
print(f"{'n':>3s} {'k':>3s} {'k^n':>18s} {'n·k^n':>20s}")
print("-" * 48)

k = N_FRANJAS
for n in range(1, N_PARTIDOS + 1):
    comb = k ** n
    ops  = n * comb
    print(f"{n:>3d} {k:>3d} {comb:>18,} {ops:>20,}")

# Verificación empírica con los tiempos medidos en cell-23
print(f"\n--- Verificación empírica (cell-23) ---")
t_medido = t1 - t0  # tiempo medido para n=6
ops_n6 = n_reducido * (N_FRANJAS ** n_reducido)
ops_n10 = N_PARTIDOS * (N_FRANJAS ** N_PARTIDOS)
print(f"n=6:  {ops_n6:,} operaciones → {t_medido:.2f} s medidos")
print(f"n=10: {ops_n10:,} operaciones → factor ×{ops_n10/ops_n6:,.0f} respecto a n=6")
print(f"      Tiempo estimado: {t_medido * (ops_n10/ops_n6):.0f} s "
      f"({t_medido * (ops_n10/ops_n6) / 3600:.1f} h)")
```

```
Crecimiento de la complejidad temporal T(n,k) = n · k^n
  n   k                k^n                n·k^n
------------------------------------------------
  1  10                 10                   10
  2  10                100                  200
  3  10              1,000                3,000
  4  10             10,000               40,000
  5  10            100,000              500,000
  6  10          1,000,000            6,000,000
  7  10         10,000,000           70,000,000
  8  10        100,000,000          800,000,000
  9  10      1,000,000,000        9,000,000,000
 10  10     10,000,000,000      100,000,000,000

--- Verificación empírica (cell-23) ---
n=6:  6,000,000 operaciones → 0.33 s medidos
n=10: 100,000,000,000 operaciones → factor ×16,667 respecto a n=6
      Tiempo estimado: 5489 s (1.5 h)
```

(*)Diseña un algoritmo que mejore la complejidad del algortimo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta: Algoritmo Greedy (voraz)

Idea clave: En lugar de explorar las k^n combinaciones, tomar decisiones localmente óptimas de forma secuencial: asignar cada partido, uno a uno, a la franja que maximice su contribución marginal a la audiencia.

Estrategia:

1. Ordenar los partidos por audiencia base de mayor a menor ($O(n \log n)$). Los partidos más valiosos se asignan primero para que tengan más opciones disponibles con menos coincidencias.
2. Para cada partido (en ese orden), evaluar las k franjas y elegir la que maximice la contribución individual del partido:

$$\text{score}(i, f) = \text{base}_i \times \text{pond}(f) \times \text{penal}(\text{partidos_ya_en_f})$$

3. Garantizar restricciones durante la asignación: si quedan pocos partidos por asignar y aún falta viernes o lunes, restringir las franjas candidatas para asegurar factibilidad.

Pseudocódigo:

```
función GREEDY(categorías, n_partidos, n_franjas):
    orden <- ordenar partidos por audiencia base DESCENDENTE
    solución <- vector vacío de tamaño n
    conteo  <- [0] × k                  // partidos asignados por franja
    tiene_viernes <- falso
    tiene_lunes   <- falso

    PARA cada partido idx EN orden:
        mejor_franja <- -1
        mejor_score  <- -∞

        PARA cada franja f DE 0 A k-1:
            SI asignar f impide cumplir restricciones futuras:
                CONTINUAR
            score ← base[idx] × pond[f] × penal[conteo[f]]
            SI score > mejor_score:
                mejor_score  <- score
                mejor_franja <- f

        solución[idx] <- mejor_franja
        conteo[mejor_franja] += 1
        actualizar tiene_viernes / tiene_lunes

    DEVOLVER solución
```

¿Por qué mejora la fuerza bruta?

Aspecto	Fuerza bruta	Greedy
Estrategia	Enumera todas las k^n soluciones	Construye una solución en n pasos
Complejidad temporal	$O(n \cdot k^n)$, exponencial	$O(n \cdot k + n \log n)$ polinomial
Para $n = k = 10$	10^{11} operaciones (horas)	$\approx 100 + 33 \approx 133$ operaciones (microsegundos)
Optimalidad	Garantizada (explora todo)	No garantizada (decisiones locales, irrevocables)

El cambio de paradigma es fundamental: pasamos de exploración exhaustiva a construcción incremental, sacrificando la garantía de optimalidad a cambio de una reducción de complejidad de exponencial a polinomial. Para nuestro problema con $n = k = 10$, esto significa pasar de horas de cómputo a microsegundos.

Limitación: Al tomar decisiones irrevocables basadas en información local, el greedy puede quedar atrapado en un óptimo local. No obstante, la heurística de asignar primero los partidos más valiosos produce soluciones de buena calidad en la práctica.

In [66]:
```python
def greedy_laliga(categorias, n_partidos, n_franjas):
    """Greedy: asigna partidos por orden de importancia a la mejor franja."""

    # Paso 1: ordenar por audiencia base descendente, O(n log n)
    orden = sorted(range(n_partidos),
                   key=lambda i: BASE_AUDIENCIA[categorias[i]], reverse=True)

    solucion = [0] * n_partidos
    conteo = [0] * n_franjas          # partidos ya asignados por franja
    tiene_viernes = False
    tiene_lunes = False

    # Paso 2: asignación greedy, O(n · k)
    for paso, idx in enumerate(orden):
        restantes_despues = n_partidos - paso - 1
        falta_v = not tiene_viernes
        falta_l = not tiene_lunes

        mejor_franja = -1
        mejor_score = -float('inf')

        for f in range(n_franjas):
            # ¿Asignar f deja suficientes huecos para cumplir restricciones?
            need_v = falta_v and (f != IDX_VIERNES)
            need_l = falta_l and (f != IDX_LUNES)
            if int(need_v) + int(need_l) > restantes_despues:
                continue   # no quedan partidos suficientes → saltar

            # Score: contribución individual del partido en esta franja
            base = BASE_AUDIENCIA[categorias[idx]]
            score = base * PONDERACION[f] * PENALIZACION[conteo[f]]

            if score > mejor_score:
                mejor_score = score
                mejor_franja = f

        solucion[idx] = mejor_franja
        conteo[mejor_franja] += 1
        if mejor_franja == IDX_VIERNES: tiene_viernes = True
        if mejor_franja == IDX_LUNES:   tiene_lunes = True

    return solucion
```

```python
# === Ejecución con los 10 partidos completos ===
tg0 = time.time()
sol_greedy = greedy_laliga(categorias_ejemplo, N_PARTIDOS, N_FRANJAS)
tg1 = time.time()

aud_greedy = funcion_objetivo(sol_greedy, categorias_ejemplo)
valida_greedy = IDX_VIERNES in sol_greedy and IDX_LUNES in sol_greedy

print("=== Algoritmo Greedy (10 partidos) ===")
print(f"Tiempo: {(tg1-tg0)*1_000_000:.0f} µs")
print(f"Solución válida: {valida_greedy}")
print(f"Audiencia total: {aud_greedy:.4f} millones")
print(f"Solución: {sol_greedy}\n")
for i in range(N_PARTIDOS):
    print(f"  P{i} ({partidos[i]:>22s}) -> {IDX_A_FRANJA[sol_greedy[i]]}")

# === Comparación con fuerza bruta (ejemplo reducido n=6) ===
tg0_6 = time.time()
sol_greedy_6 = greedy_laliga(cats_reducido, n_reducido, N_FRANJAS)
tg1_6 = time.time()
aud_greedy_6 = funcion_objetivo(sol_greedy_6, cats_reducido)

print(f"\n=== Comparación n={n_reducido} partidos ===")
print(f"{'':30s} {'Audiencia':>12s} {'Tiempo':>12s}")
print(f"{'Fuerza bruta':30s} {aud:>10.4f} M {t1-t0:>10.2f} s")
print(f"{'Greedy':30s} {aud_greedy_6:>10.4f} M {(tg1_6-tg0_6)*1e6:>7.0f} µs")
print(f"\nDiferencia audiencia: {aud_greedy_6 - aud:+.4f} M "
      f"({(aud_greedy_6 / aud - 1) * 100:+.1f}%)")
print(f"Speedup: ×{(t1-t0) / (tg1_6-tg0_6):,.0f}")
```

```
=== Algoritmo Greedy (10 partidos) ===
Tiempo: 58 µs
Solución válida: True
Audiencia total: 6.5885 millones
Solución: [4, 8, 0, 7, 9, 8, 3, 4, 2, 6]

  P0 (         Celta-RMadrid) -> S20
  P1 (    Valencia-RSociedad) -> D20
  P2 (        Mallorca-Eibar) -> V20
  P3 (    Athletic-Barcelona) -> D18
  P4 (       Leganés-Osasuna) -> L20
  P5 (    Villarreal-Granada) -> D20
  P6 (        Alavés-Levante) -> S18
  P7 (      Espanyol-Sevilla) -> S20
  P8 (      Betis-Valladolid) -> S16
  P9 (       Atlético-Getafe) -> D16

=== Comparación n=6 partidos ===
                               Audiencia       Tiempo
Fuerza bruta                    4.6810 M       0.33 s
Greedy                          4.6810 M      44 µs

Diferencia audiencia: +0.0000 M (+0.0%)
Speedup: ×7,467
```

(*)Calcula la complejidad del algoritmo

Respuesta

El algoritmo greedy tiene dos fases secuenciales:

Fase	Operación	Complejidad
1. Ordenación	Ordenar n partidos por audiencia base (Timsort)	$O(n \log n)$
2. Asignación	Para cada partido (n), evaluar k franjas	$O(n \cdot k)$

Complejidad temporal:

$$T(n,k) = O(n \log n + n \cdot k) = O(n \cdot k)$$

El término $n \cdot k$ domina sobre $n \log n$ cuando $k \geq \log n$ (en nuestro caso $k = 10 > \log_2 10 \approx 3.3$).

Con los valores del problema ($n = k = 10$):

$$T(10,10) = O(10 \cdot 10) = O(100)$$

Complejidad espacial:

$$S(n,k) = O(n + k)$$

- $O(n)$: vector solución + lista de índices ordenados.
- $O(k)$: vector de conteo de partidos por franja.

Comparación con fuerza bruta:

	Fuerza bruta	Greedy
Temporal	$O(n \cdot k^n)$, exponencial	$O(n \cdot k)$, polinomial
Espacial	$O(n + k)$	$O(n + k)$
Para $n = k = 10$	10^{11}	10^2

La mejora es de exponencial a polinomial (concretamente cuadrática cuando $k = n$): pasamos de $O(n \cdot k^n)$ a $O(n \cdot k)$, eliminando el factor exponencial k^n y sustituyéndolo por el factor lineal k. Esto representa una reducción del orden de k^{n-1} (10^9 para nuestro problema).

In [67]:
```python
# --- Complejidad del algoritmo greedy ---
# T(n,k) = O(n·k)    |    S(n,k) = O(n + k)
```

```python
import math

print("Comparación de complejidad: Fuerza bruta vs Greedy")
print(f"{'n':>3s} {'k':>3s} {'FB: n·k^n':>20s} {'Greedy: n·k':>12s} {'Reducción':>12s}")
print("-" * 53)

k = N_FRANJAS
for n in range(1, N_PARTIDOS + 1):
    fb = n * k**n
    gr = n * k
    print(f"{n:>3d} {k:>3d} {fb:>20,} {gr:>12,} {fb/gr:>11,.0f}×")

# Verificación empírica: tiempo greedy vs fuerza bruta (n=6)
print(f"\n--- Verificación empírica (n={n_reducido}) ---")
t_fb = t1 - t0                    # fuerza bruta (cell-23)
t_gr = tg1_6 - tg0_6              # greedy (cell-31)
print(f"Fuerza bruta: {t_fb:.4f} s")
print(f"Greedy:       {t_gr*1e6:.0f} µs ({t_gr:.6f} s)")
print(f"Speedup real: ×{t_fb/t_gr:,.0f}")

# Operaciones teóricas
ops_fb = n_reducido * N_FRANJAS**n_reducido
ops_gr = n_reducido * N_FRANJAS + n_reducido * math.ceil(math.log2(n_reducido))
print(f"\nOperaciones teóricas (n={n_reducido}):")
print(f"  Fuerza bruta: n·k^n = {ops_fb:,}")
print(f"  Greedy: n·k + n·log n ≈ {ops_gr}")
print(f"  Ratio teórico: ×{ops_fb/ops_gr:,.0f}")
```

```
Comparación de complejidad: Fuerza bruta vs Greedy
  n   k            FB: n·k^n  Greedy: n·k    Reducción
-----------------------------------------------------
  1  10                   10           10           1×
  2  10                  200           20          10×
  3  10                3,000           30         100×
  4  10               40,000           40       1,000×
  5  10              500,000           50      10,000×
  6  10            6,000,000           60     100,000×
  7  10           70,000,000           70   1,000,000×
  8  10          800,000,000           80  10,000,000×
  9  10        9,000,000,000           90 100,000,000×
 10  10      100,000,000,000          100 1,000,000,000×

--- Verificación empírica (n=6) ---
Fuerza bruta: 0.3294 s
Greedy:       44 µs (0.000044 s)
Speedup real: ×7,467

Operaciones teóricas (n=6):
  Fuerza bruta: n·k^n = 6,000,000
  Greedy: n·k + n·log n ≈ 78
  Ratio teórico: ×76,923
```

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

¿Qué datos de entrada necesita el algoritmo?

El único dato variable entre jornadas es el vector de **categorías de los partidos**: una lista de n tuplas `(cat_local, cat_visitante)`. Las franjas horarias, ponderaciones y penalizaciones son constantes del problema.

¿Cómo generar jornadas aleatorias realistas?

Según el enunciado, La Liga tiene **20 equipos** con la siguiente distribución de categorías:

- **3** equipos de categoría A
- **11** equipos de categoría B
- **6** equipos de categoría C

Una jornada consiste en emparejar aleatoriamente los 20 equipos en 10 partidos. El procedimiento es:

1. Crear la lista de 20 equipos con la distribución real: `["A"]*3 + ["B"]*11 + ["C"]*6`
2. **Barajar** la lista aleatoriamente (`random.shuffle`)
3. Formar **10 parejas** consecutivas: `(equipos[0], equipos[1])`, `(equipos[2], equipos[3])`, ...
4. Extraer las **categorías** de cada partido como tupla `(cat_local, cat_visitante)`

Este método respeta la distribución real de categorías y genera emparejamientos distintos en cada ejecución. Usamos `random.seed()` para reproducibilidad.

Generación de múltiples jornadas:

Para evaluar la robustez del algoritmo greedy, generamos varias jornadas aleatorias y comparamos resultados.

In [68]:
```python
import random

# Distribución real de La Liga: 3A + 11B + 6C = 20 equipos
EQUIPOS = ["A"] * 3 + ["B"] * 11 + ["C"] * 6

def generar_jornada(seed=None):
    """Genera una jornada aleatoria emparejando los 20 equipos."""
    rng = random.Random(seed)
    equipos = EQUIPOS[:]
    rng.shuffle(equipos)
    # Formar 10 parejas consecutivas
    categorias = [(equipos[2*i], equipos[2*i+1]) for i in range(10)]
    return categorias

# --- Generar 5 jornadas de ejemplo ---
N_JORNADAS = 5

print(f"Distribución de equipos: {len(EQUIPOS)} equipos "
      f"(A={EQUIPOS.count('A')}, B={EQUIPOS.count('B')}, C={EQUIPOS.count('C')})")
print(f"\n{'Jornada':>8s}  {'Categorías de los 10 partidos':40s}  {'Bases':>6s}")
```

```
print("-" * 72)

jornadas = []
for j in range(N_JORNADAS):
    cats = generar_jornada(seed=42 + j)
    jornadas.append(cats)
    bases = [BASE_AUDIENCIA[c] for c in cats]
    cats_str = " ".join(f"{a}-{b}" for a, b in cats)
    print(f"  J{j:02d}     {cats_str:40s}  {sum(bases):5.2f} M")

# --- Verificar distribución de tipos de partido ---
print("\nDistribución de tipos de partido (5 jornadas, 50 partidos):")
tipo_conteo = {}
for cats in jornadas:
    for c in cats:
        clave = f"{c[0]}-{c[1]}"
        tipo_conteo[clave] = tipo_conteo.get(clave, 0) + 1

for tipo in sorted(tipo_conteo, key=lambda t: tipo_conteo[t], reverse=True):
    print(f"  {tipo}: {tipo_conteo[tipo]} partidos "
          f"(base = {BASE_AUDIENCIA[tuple(tipo.split('-'))]:.2f} M)")
```

```
Distribución de equipos: 20 equipos (A=3, B=11, C=6)

 Jornada  Categorías de los 10 partidos              Bases
------------------------------------------------------------------------
  J00     C-B C-B B-B C-C B-B C-B A-B A-C B-B A-B    9.02 M
  J01     B-B B-A B-B C-A C-C B-C B-C B-B C-B B-A    9.02 M
  J02     B-C C-B A-B B-B B-A C-A B-C B-B B-C C-B    9.15 M
  J03     C-B B-C C-B B-C B-B B-A A-B B-A C-C B-B    9.17 M
  J04     B-C B-C C-C B-B B-C B-A C-B B-B B-A B-A    9.17 M

Distribución de tipos de partido (5 jornadas, 50 partidos):
  B-B: 12 partidos (base = 0.90 M)
  B-C: 10 partidos (base = 0.75 M)
  C-B: 9 partidos (base = 0.75 M)
  B-A: 8 partidos (base = 1.30 M)
  C-C: 4 partidos (base = 0.47 M)
  A-B: 4 partidos (base = 1.30 M)
  C-A: 2 partidos (base = 1.00 M)
  A-C: 1 partidos (base = 1.00 M)
```

Aplica el algoritmo al juego de datos generado

Respuesta

Aplicamos el algoritmo **greedy** a las 5 jornadas aleatorias generadas anteriormente. Para la primera jornada (J00), también ejecutamos **fuerza bruta con n=6 partidos** para comparar la calidad de la solución greedy frente al óptimo conocido.

Esto nos permite evaluar:

1. **Rendimiento temporal** del greedy en múltiples jornadas distintas
2. **Calidad de la solución** greedy vs óptimo exacto (en el subconjunto reducido)
3. **Consistencia**: ¿produce soluciones válidas y de buena calidad en todas las jornadas?

In [69]:

```
# === Greedy sobre las 5 jornadas aleatorias ===
print("=== Greedy aplicado a jornadas aleatorias ===\n")
print(f"{'Jornada':>8s}  {'Audiencia':>10s}  {'Tiempo':>10s}  {'Válida':>7s}  {'Solución'}")
print("-" * 75)

resultados_greedy = []
for j, cats in enumerate(jornadas):
    tg_start = time.time()
    sol_j = greedy_laliga(cats, N_PARTIDOS, N_FRANJAS)
    tg_end = time.time()

    aud_j = funcion_objetivo(sol_j, cats)
    valida_j = IDX_VIERNES in sol_j and IDX_LUNES in sol_j
    resultados_greedy.append((sol_j, aud_j, tg_end - tg_start))

    print(f"  J{j:02d}     {aud_j:>8.4f} M  {(tg_end-tg_start)*1e6:>7.0f} µs  "
          f"{'Sí':>7s}  {sol_j}" if valida_j else
          f"  J{j:02d}     {aud_j:>8.4f} M  {(tg_end-tg_start)*1e6:>7.0f} µs  "
          f"{'NO':>7s}  {sol_j}")

audiencias = [r[1] for r in resultados_greedy]
print(f"\nAudiencia media:  {sum(audiencias)/len(audiencias):.4f} M")
print(f"Audiencia mínima: {min(audiencias):.4f} M")
print(f"Audiencia máxima: {max(audiencias):.4f} M")

# === Detalle de la mejor jornada ===
idx_mejor = audiencias.index(max(audiencias))
print(f"\n--- Detalle J{idx_mejor:02d} (mayor audiencia) ---")
sol_mejor, aud_mejor, _ = resultados_greedy[idx_mejor]
cats_mejor = jornadas[idx_mejor]
for i in range(N_PARTIDOS):
    cat_str = f"{cats_mejor[i][0]}-{cats_mejor[i][1]}"
    base = BASE_AUDIENCIA[cats_mejor[i]]
    print(f"  P{i} ({cat_str})  base={base:.2f} -> {IDX_A_FRANJA[sol_mejor[i]]}")

# === Comparación Greedy vs Fuerza Bruta (J00, n=6) ===
print(f"\n{'='*55}")
print(f"=== Greedy vs Fuerza Bruta, J00, primeros 6 partidos ===")
print(f"{'='*55}")

cats_j00 = jornadas[0]
cats_j00_6 = cats_j00[:6]

# Fuerza bruta (óptimo exacto)
tfb_start = time.time()
sol_fb_j00, aud_fb_j00, ev_fb_j00 = fuerza_bruta(cats_j00_6, 6, N_FRANJAS)
tfb_end = time.time()

# Greedy
tgr_start = time.time()
```

```python
sol_gr_j00 = greedy_laliga(cats_j00_6, 6, N_FRANJAS)
tgr_end = time.time()
aud_gr_j00 = funcion_objetivo(sol_gr_j00, cats_j00_6)

print(f"\n{'':25s} {'Audiencia':>10s} {'Tiempo':>12s}")
print(f"{'Fuerza bruta (óptimo)':25s} {aud_fb_j00:>8.4f} M {tfb_end-tfb_start:>10.2f} s")
print(f"{'Greedy':25s} {aud_gr_j00:>8.4f} M {(tgr_end-tgr_start)*1e6:>7.0f} µs")

gap = (aud_fb_j00 - aud_gr_j00) / aud_fb_j00 * 100
print(f"\nGap greedy vs óptimo: {gap:.2f}%")
if gap == 0:
    print("El greedy encontró la solución óptima.")
else:
    print(f"El greedy se desvía {gap:.2f}% del óptimo exacto.")
```

```
=== Greedy aplicado a jornadas aleatorias ===

 Jornada   Audiencia      Tiempo   Válida  Solución
----------------------------------------------------------------------------
  J00       6.4455 M       61 µs       Sí  [8, 2, 3, 9, 4, 0, 4, 7, 6, 8]
  J01       6.4455 M       24 µs       Sí  [3, 4, 4, 7, 9, 8, 2, 6, 0, 8]
  J02       6.4450 M       23 µs       Sí  [6, 8, 4, 3, 8, 7, 2, 4, 0, 9]
  J03       6.5880 M       22 µs       Sí  [6, 8, 2, 0, 3, 4, 8, 7, 9, 4]
  J04       6.5880 M       21 µs       Sí  [6, 8, 9, 3, 2, 4, 0, 4, 8, 7]

Audiencia media:  6.5024 M
Audiencia mínima: 6.4450 M
Audiencia máxima: 6.5880 M

--- Detalle J04 (mayor audiencia) ---
  P0 (B-C)  base=0.75 -> D16
  P1 (B-C)  base=0.75 -> D20
  P2 (C-C)  base=0.47 -> L20
  P3 (B-B)  base=0.90 -> S18
  P4 (B-C)  base=0.75 -> S16
  P5 (B-A)  base=1.30 -> S20
  P6 (C-B)  base=0.75 -> V20
  P7 (B-B)  base=0.90 -> S20
  P8 (B-A)  base=1.30 -> D20
  P9 (B-A)  base=1.30 -> D18

=====================================================
=== Greedy vs Fuerza Bruta, J00, primeros 6 partidos ===
=====================================================

                          Audiencia       Tiempo
Fuerza bruta (óptimo)      3.5255 M       0.34 s
Greedy                     3.5255 M      47 µs

Gap greedy vs óptimo: 0.00%
El greedy encontró la solución óptima.
```

Generar calendario de partidos para todo el año

Respuesta

Datos reales de La Liga:

Dato	Valor
Equipos	20
Categorías	A=3, B=11, C=6
Partidos por jornada	10
Jornadas por temporada	38 (ida + vuelta)
Total partidos	**380**

Enfoque: Reutilizamos la función `generar_jornada(seed)` ya definida para generar 38 jornadas aleatorias (una por cada semilla distinta), respetando la distribución real de categorías. Luego aplicamos el algoritmo **greedy** a cada jornada para obtener la programación horaria de toda la temporada.

In [70]:
```python
# === Calendario completo: 38 jornadas × 10 partidos = 380 partidos ===

N_JORNADAS_TEMPORADA = 38
DIAS = {"V": "Viernes", "S": "Sábado", "D": "Domingo", "L": "Lunes"}

# --- Equipos con nombre y categoría ---
LIGA_EQUIPOS = [
    ("Real Madrid",    "A"), ("Barcelona",     "A"), ("Atlético",       "A"),
    ("Athletic",       "B"), ("R. Sociedad",   "B"), ("Betis",          "B"),
    ("Villarreal",     "B"), ("Sevilla",        "B"), ("Valencia",       "B"),
    ("Celta",          "B"), ("Espanyol",       "B"), ("Osasuna",        "B"),
    ("Mallorca",       "B"), ("Rayo Vallecano", "B"),
    ("Alavés",         "C"), ("Valladolid",     "C"), ("Las Palmas",     "C"),
    ("Leganés",        "C"), ("Eibar",          "C"), ("Getafe",         "C"),
]

def generar_jornada_completa(seed):
    """Genera una jornada aleatoria con nombres, categorías y emparejamientos."""
    rng = random.Random(seed)
    equipos = LIGA_EQUIPOS[:]
    rng.shuffle(equipos)
    nombres_j = []
    categorias_j = []
    for i in range(0, len(equipos), 2):
        local, cat_l = equipos[i]
        visitante, cat_v = equipos[i + 1]
        nombres_j.append(f"{local}-{visitante}")
        categorias_j.append((cat_l, cat_v))
    return categorias_j, nombres_j

# --- Generar 38 jornadas y aplicar greedy a cada una ---
temporada = []
```

```python
t_total_start = time.time()
for j in range(N_JORNADAS_TEMPORADA):
    cats_j, nombres_j = generar_jornada_completa(seed=100 + j)
    sol_j = greedy_laliga(cats_j, N_PARTIDOS, N_FRANJAS)
    aud_j = funcion_objetivo(sol_j, cats_j)
    temporada.append((cats_j, sol_j, aud_j, nombres_j))
t_total_end = time.time()

audiencias_temp = [t[2] for t in temporada]

print(f"Temporada: {N_JORNADAS_TEMPORADA} jornadas, "
      f"{N_JORNADAS_TEMPORADA * N_PARTIDOS} partidos")
print(f"Greedy aplicado en {(t_total_end - t_total_start)*1000:.1f} ms\n")
print(f"Audiencia total temporada: {sum(audiencias_temp):.2f} millones")
print(f"Audiencia media/jornada:   {sum(audiencias_temp)/len(audiencias_temp):.4f} millones")
print(f"Mayor audiencia: J{audiencias_temp.index(max(audiencias_temp))+1:02d} "
      f"({max(audiencias_temp):.4f} M)")
print(f"Menor audiencia: J{audiencias_temp.index(min(audiencias_temp))+1:02d} "
      f"({min(audiencias_temp):.4f} M)")

# === Gráfico de barras de audiencia por jornada ===
print(f"\n{'AUDIENCIA POR JORNADA':^60s}")
print(f"{'─'*60}")

max_aud = max(audiencias_temp)
for j in range(N_JORNADAS_TEMPORADA):
    aud = audiencias_temp[j]
    barra_len = int(aud / max_aud * 35)
    barra = "▇" * barra_len
    print(f"  J{j+1:02d}  {aud:>7.4f} M  ▌{barra}")

# === Detalle de TODAS las jornadas ===
print(f"\n{'='*78}")
print(f"{'CALENDARIO LA LIGA, PROGRAMACIÓN HORARIA COMPLETA':^78s}")
print(f"{'='*78}")

for j in range(N_JORNADAS_TEMPORADA):
    cats_j, sol_j, aud_j, nombres_j = temporada[j]
    print(f"\n┌─ Jornada {j+1:02d} {'─'*46} Audiencia: {aud_j:.4f} M ─┐")

    # Agrupar por día
    por_dia = {}
    for i in range(N_PARTIDOS):
        franja = IDX_A_FRANJA[sol_j[i]]
        dia = franja[0]
        hora = franja[1:]
        cat_str = f"{cats_j[i][0]}-{cats_j[i][1]}"
        base = BASE_AUDIENCIA[cats_j[i]]
        if dia not in por_dia:
            por_dia[dia] = []
        por_dia[dia].append((hora, nombres_j[i], cat_str, base))

    for dia_cod in ["V", "S", "D", "L"]:
        if dia_cod not in por_dia:
            continue
        partidos_dia = sorted(por_dia[dia_cod], key=lambda x: x[0])
        for k, (hora, nombre, cat, base) in enumerate(partidos_dia):
            dia_label = DIAS[dia_cod] if k == 0 else ""
            print(f"│  {dia_label:8s}  {hora}h  {nombre:<28s} ({cat}, {base:.2f} M)")

    print(f"└{'─'*76}┘")
```

```
Temporada: 38 jornadas, 380 partidos
Greedy aplicado en 1.3 ms

Audiencia total temporada: 245.91 millones
Audiencia media/jornada:   6.4714 millones
Mayor audiencia: J26 (6.7730 M)
Menor audiencia: J31 (6.3325 M)

                    AUDIENCIA POR JORNADA
──────────────────────────────────────────────────────────────
  J01   6.4450 M  ▌
  J02   6.6600 M  ▌
  J03   6.4450 M  ▌
  J04   6.4450 M  ▌
  J05   6.4460 M  ▌
  J06   6.3330 M  ▌
  J07   6.4460 M  ▌
  J08   6.5880 M  ▌
  J09   6.4455 M  ▌
  J10   6.4455 M  ▌
  J11   6.3325 M  ▌
  J12   6.4455 M  ▌
  J13   6.4460 M  ▌
  J14   6.5880 M  ▌
  J15   6.5880 M  ▌
  J16   6.3330 M  ▌
  J17   6.5885 M  ▌
  J18   6.3330 M  ▌
  J19   6.5880 M  ▌
  J20   6.4455 M  ▌
  J21   6.4450 M  ▌
  J22   6.5880 M  ▌
  J23   6.3330 M  ▌
  J24   6.4455 M  ▌
  J25   6.5885 M  ▌
  J26   6.7730 M  ▌
  J27   6.4455 M  ▌
  J28   6.3325 M  ▌
  J29   6.6600 M  ▌
  J30   6.3330 M  ▌
  J31   6.3325 M  ▌
  J32   6.4455 M  ▌
  J33   6.3325 M  ▌
  J34   6.5880 M  ▌
  J35   6.4455 M  ▌
  J36   6.6600 M  ▌
  J37   6.3325 M  ▌
  J38   6.4455 M  ▌

==============================================================================
              CALENDARIO LA LIGA, PROGRAMACIÓN HORARIA COMPLETA
==============================================================================

┌─ Jornada 01 ──────────────────────────────────────────── Audiencia: 6.4450 M ─┐
│  Viernes   20h  Betis-Eibar                  (B-C, 0.75 M)
│  Sábado    16h  Villarreal-Las Palmas        (B-C, 0.75 M)
│            18h  Rayo Vallecano-Celta         (B-B, 0.90 M)
│            20h  Atlético-Athletic            (A-B, 1.30 M)
│            20h  Osasuna-Mallorca             (B-B, 0.90 M)
│  Domingo   16h  Sevilla-Valladolid           (B-C, 0.75 M)
│            18h  Getafe-Real Madrid           (C-A, 1.00 M)
│            20h  Espanyol-Leganés             (B-C, 0.75 M)
│            20h  Barcelona-Valencia           (A-B, 1.30 M)
│  Lunes     20h  Alavés-R. Sociedad           (C-B, 0.75 M)
└──────────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 02 ──────────────────────────────────────────── Audiencia: 6.6600 M ─┐
│  Viernes   20h  Osasuna-Leganés              (B-C, 0.75 M)
│  Sábado    16h  R. Sociedad-Las Palmas       (B-C, 0.75 M)
│            18h  Celta-Athletic               (B-B, 0.90 M)
│            20h  Atlético-Barcelona           (A-A, 2.00 M)
│            20h  Valencia-Espanyol            (B-B, 0.90 M)
│  Domingo   16h  Mallorca-Getafe              (B-C, 0.75 M)
│            18h  Betis-Sevilla                (B-B, 0.90 M)
│            20h  Valladolid-Rayo Vallecano    (C-B, 0.75 M)
│            20h  Real Madrid-Alavés           (A-C, 1.00 M)
│  Lunes     20h  Villarreal-Eibar             (B-C, 0.75 M)
└──────────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 03 ──────────────────────────────────────────── Audiencia: 6.4450 M ─┐
│  Viernes   20h  Getafe-Mallorca              (C-B, 0.75 M)
│  Sábado    16h  Eibar-Rayo Vallecano         (C-B, 0.75 M)
│            18h  Villarreal-Celta             (B-B, 0.90 M)
│            20h  Sevilla-Barcelona            (B-A, 1.30 M)
│            20h  Osasuna-R. Sociedad          (B-B, 0.90 M)
│  Domingo   16h  Alavés-Espanyol              (C-B, 0.75 M)
│            18h  Las Palmas-Atlético          (C-A, 1.00 M)
│            20h  Valladolid-Athletic          (C-B, 0.75 M)
│            20h  Valencia-Real Madrid         (B-A, 1.30 M)
│  Lunes     20h  Leganés-Betis                (C-B, 0.75 M)
└──────────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 04 ──────────────────────────────────────────── Audiencia: 6.4450 M ─┐
│  Viernes   20h  Leganés-Osasuna              (C-B, 0.75 M)
│  Sábado    16h  R. Sociedad-Las Palmas       (B-C, 0.75 M)
│            18h  Espanyol-Betis               (B-B, 0.90 M)
│            20h  Rayo Vallecano-Real Madrid   (B-A, 1.30 M)
│            20h  Sevilla-Mallorca             (B-B, 0.90 M)
│  Domingo   16h  Valladolid-Valencia          (C-B, 0.75 M)
│            18h  Getafe-Barcelona             (C-A, 1.00 M)
│            20h  Eibar-Celta                  (C-B, 0.75 M)
│            20h  Athletic-Atlético            (B-A, 1.30 M)
│  Lunes     20h  Villarreal-Alavés            (B-C, 0.75 M)
└──────────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 05 ──────────────────────────────────────────── Audiencia: 6.4460 M ─┐
│  Viernes   20h  Leganés-Eibar                (C-C, 0.47 M)
│  Sábado    16h  Getafe-Espanyol              (C-B, 0.75 M)
│            18h  Athletic-R. Sociedad         (B-B, 0.90 M)
│            20h  Valencia-Betis               (B-B, 0.90 M)
```

	20h	Atlético-Osasuna	(A-B, 1.30 M)
Domingo	16h	Rayo Vallecano-Celta	(B-B, 0.90 M)
	18h	Valladolid-Barcelona	(C-A, 1.00 M)
	20h	Mallorca-Sevilla	(B-B, 0.90 M)
	20h	Villarreal-Real Madrid	(B-A, 1.30 M)
Lunes	20h	Las Palmas-Alavés	(C-C, 0.47 M)

Jornada 06 — Audiencia: 6.3330 M

Viernes	20h	Osasuna-Eibar	(B-C, 0.75 M)
Sábado	16h	Valladolid-Mallorca	(C-B, 0.75 M)
	18h	Sevilla-Espanyol	(B-B, 0.90 M)
	20h	Villarreal-Rayo Vallecano	(B-B, 0.90 M)
	20h	Real Madrid-Athletic	(A-B, 1.30 M)
Domingo	16h	R. Sociedad-Betis	(B-B, 0.90 M)
	18h	Getafe-Atlético	(C-A, 1.00 M)
	20h	Las Palmas-Barcelona	(C-A, 1.00 M)
	20h	Valencia-Celta	(B-B, 0.90 M)
Lunes	20h	Alavés-Leganés	(C-C, 0.47 M)

Jornada 07 — Audiencia: 6.4460 M

Viernes	20h	Eibar-Getafe	(C-C, 0.47 M)
Sábado	16h	Alavés-Celta	(C-B, 0.75 M)
	18h	Rayo Vallecano-Osasuna	(B-B, 0.90 M)
	20h	Atlético-Valencia	(A-B, 1.30 M)
	20h	Espanyol-Villarreal	(B-B, 0.90 M)
Domingo	16h	Betis-Sevilla	(B-B, 0.90 M)
	18h	Real Madrid-Valladolid	(A-C, 1.00 M)
	20h	Athletic-Barcelona	(B-A, 1.30 M)
	20h	R. Sociedad-Mallorca	(B-B, 0.90 M)
Lunes	20h	Las Palmas-Leganés	(C-C, 0.47 M)

Jornada 08 — Audiencia: 6.5880 M

Viernes	20h	Eibar-Sevilla	(C-B, 0.75 M)
Sábado	16h	Valladolid-Rayo Vallecano	(C-B, 0.75 M)
	18h	Athletic-Mallorca	(B-B, 0.90 M)
	20h	Real Madrid-Espanyol	(A-B, 1.30 M)
	20h	Valencia-Villarreal	(B-B, 0.90 M)
Domingo	16h	Betis-Getafe	(B-C, 0.75 M)
	18h	Atlético-Celta	(A-B, 1.30 M)
	20h	Osasuna-Alavés	(B-C, 0.75 M)
	20h	R. Sociedad-Barcelona	(B-A, 1.30 M)
Lunes	20h	Leganés-Las Palmas	(C-C, 0.47 M)

Jornada 09 — Audiencia: 6.4455 M

Viernes	20h	Athletic-Leganés	(B-C, 0.75 M)
Sábado	16h	Valladolid-Sevilla	(C-B, 0.75 M)
	18h	Villarreal-Rayo Vallecano	(B-B, 0.90 M)
	20h	Real Madrid-Celta	(A-B, 1.30 M)
	20h	Betis-Osasuna	(B-B, 0.90 M)
Domingo	16h	Valencia-Mallorca	(B-B, 0.90 M)
	18h	Getafe-Barcelona	(C-A, 1.00 M)
	20h	Espanyol-Las Palmas	(B-C, 0.75 M)
	20h	Atlético-R. Sociedad	(A-B, 1.30 M)
Lunes	20h	Eibar-Alavés	(C-C, 0.47 M)

Jornada 10 — Audiencia: 6.4455 M

Viernes	20h	Getafe-Athletic	(C-B, 0.75 M)
Sábado	16h	R. Sociedad-Eibar	(B-C, 0.75 M)
	18h	Rayo Vallecano-Osasuna	(B-B, 0.90 M)
	20h	Espanyol-Real Madrid	(B-A, 1.30 M)
	20h	Mallorca-Celta	(B-B, 0.90 M)
Domingo	16h	Sevilla-Valencia	(B-B, 0.90 M)
	18h	Valladolid-Barcelona	(C-A, 1.00 M)
	20h	Betis-Atlético	(B-A, 1.30 M)
	20h	Villarreal-Las Palmas	(B-C, 0.75 M)
Lunes	20h	Leganés-Alavés	(C-C, 0.47 M)

Jornada 11 — Audiencia: 6.3325 M

Viernes	20h	Osasuna-Las Palmas	(B-C, 0.75 M)
Sábado	16h	Alavés-Villarreal	(C-B, 0.75 M)
	18h	Athletic-R. Sociedad	(B-B, 0.90 M)
	20h	Celta-Real Madrid	(B-A, 1.30 M)
	20h	Espanyol-Valencia	(B-B, 0.90 M)
Domingo	16h	Sevilla-Mallorca	(B-B, 0.90 M)
	18h	Leganés-Barcelona	(C-A, 1.00 M)
	20h	Atlético-Getafe	(A-C, 1.00 M)
	20h	Eibar-Betis	(C-B, 0.75 M)
Lunes	20h	Valladolid-Rayo Vallecano	(C-B, 0.75 M)

Jornada 12 — Audiencia: 6.4455 M

Viernes	20h	Las Palmas-Mallorca	(C-B, 0.75 M)
Sábado	16h	Leganés-Celta	(C-B, 0.75 M)
	18h	Valencia-R. Sociedad	(B-B, 0.90 M)
	20h	Real Madrid-Osasuna	(A-B, 1.30 M)
	20h	Sevilla-Betis	(B-B, 0.90 M)
Domingo	16h	Espanyol-Villarreal	(B-B, 0.90 M)
	18h	Eibar-Atlético	(C-A, 1.00 M)
	20h	Barcelona-Rayo Vallecano	(A-B, 1.30 M)
	20h	Alavés-Athletic	(C-B, 0.75 M)
Lunes	20h	Getafe-Valladolid	(C-C, 0.47 M)

Jornada 13 — Audiencia: 6.4460 M

Viernes	20h	Leganés-Las Palmas	(C-C, 0.47 M)
Sábado	16h	Alavés-Celta	(C-B, 0.75 M)
	18h	Osasuna-Villarreal	(B-B, 0.90 M)
	20h	Barcelona-R. Sociedad	(A-B, 1.30 M)
	20h	Sevilla-Valencia	(B-B, 0.90 M)
Domingo	16h	Athletic-Betis	(B-B, 0.90 M)
	18h	Getafe-Real Madrid	(C-A, 1.00 M)
	20h	Atlético-Espanyol	(A-B, 1.30 M)
	20h	Mallorca-Rayo Vallecano	(B-B, 0.90 M)
Lunes	20h	Eibar-Valladolid	(C-C, 0.47 M)

```
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 14 ──────────────────────────────────────── Audiencia: 6.5880 M ─┐
│  Viernes   20h  Leganés-Villarreal          (C-B, 0.75 M)
│  Sábado    16h  Alavés-Rayo Vallecano       (C-B, 0.75 M)
│            18h  Celta-Sevilla               (B-B, 0.90 M)
│            20h  Espanyol-Atlético           (B-A, 1.30 M)
│            20h  Athletic-Osasuna            (B-B, 0.90 M)
│  Domingo   16h  Eibar-Mallorca              (C-B, 0.75 M)
│            18h  Valencia-Real Madrid        (B-A, 1.30 M)
│            20h  Las Palmas-Betis            (C-B, 0.75 M)
│            20h  R. Sociedad-Barcelona       (B-A, 1.30 M)
│  Lunes     20h  Valladolid-Getafe           (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 15 ──────────────────────────────────────── Audiencia: 6.5880 M ─┐
│  Viernes   20h  Eibar-Sevilla               (C-B, 0.75 M)
│  Sábado    16h  Rayo Vallecano-Alavés       (B-C, 0.75 M)
│            18h  Villarreal-Mallorca         (B-B, 0.90 M)
│            20h  Barcelona-Osasuna           (A-B, 1.30 M)
│            20h  Espanyol-Athletic           (B-B, 0.90 M)
│  Domingo   16h  Leganés-Betis               (C-B, 0.75 M)
│            18h  Real Madrid-Celta           (A-B, 1.30 M)
│            20h  Valladolid-R. Sociedad      (C-B, 0.75 M)
│            20h  Valencia-Atlético           (B-A, 1.30 M)
│  Lunes     20h  Las Palmas-Getafe           (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 16 ──────────────────────────────────────── Audiencia: 6.3330 M ─┐
│  Viernes   20h  Leganés-Athletic            (C-B, 0.75 M)
│  Sábado    16h  Valladolid-R. Sociedad      (C-B, 0.75 M)
│            18h  Betis-Osasuna               (B-B, 0.90 M)
│            20h  Real Madrid-Sevilla         (A-B, 1.30 M)
│            20h  Valencia-Rayo Vallecano     (B-B, 0.90 M)
│  Domingo   16h  Mallorca-Espanyol           (B-B, 0.90 M)
│            18h  Eibar-Barcelona             (C-A, 1.00 M)
│            20h  Las Palmas-Atlético         (C-A, 1.00 M)
│            20h  Villarreal-Celta            (B-B, 0.90 M)
│  Lunes     20h  Alavés-Getafe               (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 17 ──────────────────────────────────────── Audiencia: 6.5885 M ─┐
│  Viernes   20h  Las Palmas-Eibar            (C-C, 0.47 M)
│  Sábado    16h  Valladolid-Mallorca         (C-B, 0.75 M)
│            18h  Betis-Espanyol              (B-B, 0.90 M)
│            20h  Rayo Vallecano-Barcelona    (B-A, 1.30 M)
│            20h  Villarreal-R. Sociedad      (B-B, 0.90 M)
│  Domingo   16h  Osasuna-Athletic            (B-B, 0.90 M)
│            18h  Real Madrid-Celta           (A-B, 1.30 M)
│            20h  Alavés-Valencia             (C-B, 0.75 M)
│            20h  Sevilla-Atlético            (B-A, 1.30 M)
│  Lunes     20h  Leganés-Getafe              (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 18 ──────────────────────────────────────── Audiencia: 6.3330 M ─┐
│  Viernes   20h  Villarreal-Eibar            (B-C, 0.75 M)
│  Sábado    16h  R. Sociedad-Alavés          (B-C, 0.75 M)
│            18h  Osasuna-Espanyol            (B-B, 0.90 M)
│            20h  Athletic-Celta              (B-B, 0.90 M)
│            20h  Valencia-Atlético           (B-A, 1.30 M)
│  Domingo   16h  Rayo Vallecano-Mallorca     (B-B, 0.90 M)
│            18h  Barcelona-Las Palmas        (A-C, 1.00 M)
│            20h  Real Madrid-Getafe          (A-C, 1.00 M)
│            20h  Betis-Sevilla               (B-B, 0.90 M)
│  Lunes     20h  Leganés-Valladolid          (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 19 ──────────────────────────────────────── Audiencia: 6.5880 M ─┐
│  Viernes   20h  Mallorca-Eibar              (B-C, 0.75 M)
│  Sábado    16h  Rayo Vallecano-Getafe       (B-C, 0.75 M)
│            18h  Osasuna-Athletic            (B-B, 0.90 M)
│            20h  Valencia-Atlético           (B-A, 1.30 M)
│            20h  Celta-Betis                 (B-B, 0.90 M)
│  Domingo   16h  Leganés-Espanyol            (C-B, 0.75 M)
│            18h  R. Sociedad-Real Madrid     (B-A, 1.30 M)
│            20h  Barcelona-Villarreal        (A-B, 1.30 M)
│            20h  Sevilla-Valladolid          (B-C, 0.75 M)
│  Lunes     20h  Alavés-Las Palmas           (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 20 ──────────────────────────────────────── Audiencia: 6.4455 M ─┐
│  Viernes   20h  Rayo Vallecano-Leganés      (B-C, 0.75 M)
│  Sábado    16h  Getafe-R. Sociedad          (C-B, 0.75 M)
│            18h  Sevilla-Osasuna             (B-B, 0.90 M)
│            20h  Real Madrid-Valencia        (A-B, 1.30 M)
│            20h  Espanyol-Villarreal         (B-B, 0.90 M)
│  Domingo   16h  Betis-Celta                 (B-B, 0.90 M)
│            18h  Atlético-Valladolid         (A-C, 1.00 M)
│            20h  Mallorca-Las Palmas         (B-C, 0.75 M)
│            20h  Barcelona-Athletic          (A-B, 1.30 M)
│  Lunes     20h  Eibar-Alavés                (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 21 ──────────────────────────────────────── Audiencia: 6.4450 M ─┐
│  Viernes   20h  Alavés-Villarreal           (C-B, 0.75 M)
│  Sábado    16h  Celta-Getafe                (B-C, 0.75 M)
│            18h  R. Sociedad-Rayo Vallecano  (B-B, 0.90 M)
│            20h  Osasuna-Barcelona           (B-A, 1.30 M)
│            20h  Athletic-Mallorca           (B-B, 0.90 M)
│  Domingo   16h  Leganés-Espanyol            (C-B, 0.75 M)
│            18h  Real Madrid-Valladolid      (A-C, 1.00 M)
│            20h  Eibar-Valencia              (C-B, 0.75 M)
│            20h  Betis-Atlético              (B-A, 1.30 M)
│  Lunes     20h  Sevilla-Las Palmas          (B-C, 0.75 M)
└──────────────────────────────────────────────────────────────────────────┘

┌─ Jornada 22 ──────────────────────────────────────── Audiencia: 6.5880 M ─┐
│  Viernes   20h  Getafe-R. Sociedad          (C-B, 0.75 M)
│  Sábado    16h  Eibar-Espanyol              (C-B, 0.75 M)
│            18h  Rayo Vallecano-Valencia     (B-B, 0.90 M)
```

	20h	Barcelona–Sevilla	(A–B, 1.30 M)
	20h	Betis–Mallorca	(B–B, 0.90 M)
Domingo	16h	Alavés–Athletic	(C–B, 0.75 M)
	18h	Villarreal–Atlético	(B–A, 1.30 M)
	20h	Celta–Leganés	(B–C, 0.75 M)
	20h	Osasuna–Real Madrid	(B–A, 1.30 M)
Lunes	20h	Valladolid–Las Palmas	(C–C, 0.47 M)

Jornada 23 — Audiencia: 6.3330 M

Viernes	20h	Villarreal–Las Palmas	(B–C, 0.75 M)
Sábado	16h	Leganés–Espanyol	(C–B, 0.75 M)
	18h	Valencia–Betis	(B–B, 0.90 M)
	20h	Rayo Vallecano–R. Sociedad	(B–B, 0.90 M)
	20h	Atlético–Celta	(A–B, 1.30 M)
Domingo	16h	Sevilla–Athletic	(B–B, 0.90 M)
	18h	Barcelona–Getafe	(A–C, 1.00 M)
	20h	Valladolid–Real Madrid	(C–A, 1.00 M)
	20h	Mallorca–Osasuna	(B–B, 0.90 M)
Lunes	20h	Alavés–Eibar	(C–C, 0.47 M)

Jornada 24 — Audiencia: 6.4455 M

Viernes	20h	Valladolid–Villarreal	(C–B, 0.75 M)
Sábado	16h	Alavés–Mallorca	(C–B, 0.75 M)
	18h	Athletic–R. Sociedad	(B–B, 0.90 M)
	20h	Osasuna–Celta	(B–B, 0.90 M)
	20h	Rayo Vallecano–Atlético	(B–A, 1.30 M)
Domingo	16h	Betis–Espanyol	(B–B, 0.90 M)
	18h	Real Madrid–Las Palmas	(A–C, 1.00 M)
	20h	Sevilla–Leganés	(B–C, 0.75 M)
	20h	Valencia–Barcelona	(B–A, 1.30 M)
Lunes	20h	Getafe–Eibar	(C–C, 0.47 M)

Jornada 25 — Audiencia: 6.5885 M

Viernes	20h	Getafe–Alavés	(C–C, 0.47 M)
Sábado	16h	Leganés–Valencia	(C–B, 0.75 M)
	18h	Sevilla–Celta	(B–B, 0.90 M)
	20h	Espanyol–Athletic	(B–B, 0.90 M)
	20h	Atlético–Osasuna	(A–B, 1.30 M)
Domingo	16h	Mallorca–R. Sociedad	(B–B, 0.90 M)
	18h	Betis–Real Madrid	(B–A, 1.30 M)
	20h	Rayo Vallecano–Valladolid	(B–C, 0.75 M)
	20h	Villarreal–Barcelona	(B–A, 1.30 M)
Lunes	20h	Eibar–Las Palmas	(C–C, 0.47 M)

Jornada 26 — Audiencia: 6.7730 M

Viernes	20h	Getafe–Sevilla	(C–B, 0.75 M)
Sábado	16h	R. Sociedad–Leganés	(B–C, 0.75 M)
	18h	Espanyol–Rayo Vallecano	(B–B, 0.90 M)
	20h	Barcelona–Real Madrid	(A–A, 2.00 M)
	20h	Betis–Celta	(B–B, 0.90 M)
Domingo	16h	Valencia–Valladolid	(B–C, 0.75 M)
	18h	Villarreal–Mallorca	(B–B, 0.90 M)
	20h	Athletic–Alavés	(B–C, 0.75 M)
	20h	Atlético–Osasuna	(A–B, 1.30 M)
Lunes	20h	Las Palmas–Eibar	(C–C, 0.47 M)

Jornada 27 — Audiencia: 6.4455 M

Viernes	20h	Sevilla–Alavés	(B–C, 0.75 M)
Sábado	16h	Las Palmas–Betis	(C–B, 0.75 M)
	18h	Mallorca–Valencia	(B–B, 0.90 M)
	20h	R. Sociedad–Real Madrid	(B–A, 1.30 M)
	20h	Celta–Villarreal	(B–B, 0.90 M)
Domingo	16h	Rayo Vallecano–Osasuna	(B–B, 0.90 M)
	18h	Barcelona–Eibar	(A–C, 1.00 M)
	20h	Espanyol–Atlético	(B–A, 1.30 M)
	20h	Athletic–Leganés	(B–C, 0.75 M)
Lunes	20h	Valladolid–Getafe	(C–C, 0.47 M)

Jornada 28 — Audiencia: 6.3325 M

Viernes	20h	Villarreal–Leganés	(B–C, 0.75 M)
Sábado	16h	Alavés–Rayo Vallecano	(C–B, 0.75 M)
	18h	Betis–Osasuna	(B–B, 0.90 M)
	20h	Celta–Valencia	(B–B, 0.90 M)
	20h	Real Madrid–Athletic	(A–B, 1.30 M)
Domingo	16h	Mallorca–Espanyol	(B–B, 0.90 M)
	18h	Eibar–Barcelona	(C–A, 1.00 M)
	20h	Getafe–Sevilla	(C–B, 0.75 M)
	20h	Atlético–Valladolid	(A–C, 1.00 M)
Lunes	20h	R. Sociedad–Las Palmas	(B–C, 0.75 M)

Jornada 29 — Audiencia: 6.6600 M

Viernes	20h	Getafe–R. Sociedad	(C–B, 0.75 M)
Sábado	16h	Valladolid–Valencia	(C–B, 0.75 M)
	18h	Espanyol–Betis	(B–B, 0.90 M)
	20h	Atlético–Real Madrid	(A–A, 2.00 M)
	20h	Mallorca–Sevilla	(B–B, 0.90 M)
Domingo	16h	Athletic–Alavés	(B–C, 0.75 M)
	18h	Rayo Vallecano–Celta	(B–B, 0.90 M)
	20h	Barcelona–Eibar	(A–C, 1.00 M)
	20h	Leganés–Villarreal	(C–B, 0.75 M)
Lunes	20h	Osasuna–Las Palmas	(B–C, 0.75 M)

Jornada 30 — Audiencia: 6.3330 M

Viernes	20h	Getafe–Rayo Vallecano	(C–B, 0.75 M)
Sábado	16h	Valladolid–Celta	(C–B, 0.75 M)
	18h	Villarreal–Osasuna	(B–B, 0.90 M)
	20h	R. Sociedad–Sevilla	(B–B, 0.90 M)
	20h	Barcelona–Valencia	(A–B, 1.30 M)
Domingo	16h	Betis–Athletic	(B–B, 0.90 M)
	18h	Atlético–Leganés	(A–C, 1.00 M)
	20h	Real Madrid–Alavés	(A–C, 1.00 M)
	20h	Espanyol–Mallorca	(B–B, 0.90 M)

```
│ Lunes     20h  Las Palmas-Eibar             (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 31 ──────────────────────────────────────── Audiencia: 6.3325 M ─┐
│ Viernes   20h  Alavés-Valencia              (C-B, 0.75 M)
│ Sábado    16h  Eibar-Celta                  (C-B, 0.75 M)
│           18h  Osasuna-Villarreal           (B-B, 0.90 M)
│           20h  R. Sociedad-Barcelona        (B-A, 1.30 M)
│           20h  Espanyol-Mallorca            (B-B, 0.90 M)
│ Domingo   16h  Athletic-Betis               (B-B, 0.90 M)
│           18h  Real Madrid-Getafe           (A-C, 1.00 M)
│           20h  Valladolid-Atlético          (C-A, 1.00 M)
│           20h  Sevilla-Leganés              (B-C, 0.75 M)
│ Lunes     20h  Rayo Vallecano-Las Palmas    (B-C, 0.75 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 32 ──────────────────────────────────────── Audiencia: 6.4455 M ─┐
│ Viernes   20h  Eibar-Celta                  (C-B, 0.75 M)
│ Sábado    16h  Valencia-Leganés             (B-C, 0.75 M)
│           18h  Sevilla-Rayo Vallecano       (B-B, 0.90 M)
│           20h  Betis-Barcelona              (B-A, 1.30 M)
│           20h  R. Sociedad-Mallorca         (B-B, 0.90 M)
│ Domingo   16h  Osasuna-Espanyol             (B-B, 0.90 M)
│           18h  Atlético-Las Palmas          (A-C, 1.00 M)
│           20h  Valladolid-Athletic          (C-B, 0.75 M)
│           20h  Villarreal-Real Madrid       (B-A, 1.30 M)
│ Lunes     20h  Alavés-Getafe                (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 33 ──────────────────────────────────────── Audiencia: 6.3325 M ─┐
│ Viernes   20h  Valencia-Las Palmas          (B-C, 0.75 M)
│ Sábado    16h  Athletic-Leganés             (B-C, 0.75 M)
│           18h  Celta-Mallorca               (B-B, 0.90 M)
│           20h  Espanyol-Osasuna             (B-B, 0.90 M)
│           20h  Atlético-Sevilla             (A-B, 1.30 M)
│ Domingo   16h  R. Sociedad-Rayo Vallecano   (B-B, 0.90 M)
│           18h  Barcelona-Getafe             (A-C, 1.00 M)
│           20h  Villarreal-Valladolid        (B-C, 0.75 M)
│           20h  Eibar-Real Madrid            (C-A, 1.00 M)
│ Lunes     20h  Alavés-Betis                 (C-B, 0.75 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 34 ──────────────────────────────────────── Audiencia: 6.5880 M ─┐
│ Viernes   20h  Osasuna-Valladolid           (B-C, 0.75 M)
│ Sábado    16h  Espanyol-Getafe              (B-C, 0.75 M)
│           18h  Sevilla-Betis                (B-B, 0.90 M)
│           20h  Villarreal-Athletic          (B-B, 0.90 M)
│           20h  Atlético-Rayo Vallecano      (A-B, 1.30 M)
│ Domingo   16h  Leganés-Mallorca             (C-B, 0.75 M)
│           18h  Real Madrid-Celta            (A-B, 1.30 M)
│           20h  R. Sociedad-Barcelona        (B-A, 1.30 M)
│           20h  Eibar-Valencia               (C-B, 0.75 M)
│ Lunes     20h  Alavés-Las Palmas            (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 35 ──────────────────────────────────────── Audiencia: 6.4455 M ─┐
│ Viernes   20h  Eibar-Celta                  (C-B, 0.75 M)
│ Sábado    16h  Valencia-Alavés              (B-C, 0.75 M)
│           18h  Rayo Vallecano-Athletic      (B-B, 0.90 M)
│           20h  Atlético-Osasuna             (A-B, 1.30 M)
│           20h  Sevilla-Mallorca             (B-B, 0.90 M)
│ Domingo   16h  R. Sociedad-Villarreal       (B-B, 0.90 M)
│           18h  Leganés-Barcelona            (C-A, 1.00 M)
│           20h  Betis-Getafe                 (B-C, 0.75 M)
│           20h  Espanyol-Real Madrid         (B-A, 1.30 M)
│ Lunes     20h  Las Palmas-Valladolid        (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 36 ──────────────────────────────────────── Audiencia: 6.6600 M ─┐
│ Viernes   20h  Valladolid-Valencia          (C-B, 0.75 M)
│ Sábado    16h  Alavés-Betis                 (C-B, 0.75 M)
│           18h  R. Sociedad-Sevilla          (B-B, 0.90 M)
│           20h  Atlético-Real Madrid         (A-A, 2.00 M)
│           20h  Rayo Vallecano-Celta         (B-B, 0.90 M)
│ Domingo   16h  Las Palmas-Athletic          (C-B, 0.75 M)
│           18h  Mallorca-Espanyol            (B-B, 0.90 M)
│           20h  Villarreal-Leganés           (B-C, 0.75 M)
│           20h  Barcelona-Getafe             (A-C, 1.00 M)
│ Lunes     20h  Eibar-Osasuna                (C-B, 0.75 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 37 ──────────────────────────────────────── Audiencia: 6.3325 M ─┐
│ Viernes   20h  Sevilla-Getafe               (B-C, 0.75 M)
│ Sábado    16h  Alavés-Espanyol              (C-B, 0.75 M)
│           18h  Osasuna-Celta                (B-B, 0.90 M)
│           20h  R. Sociedad-Valencia         (B-B, 0.90 M)
│           20h  Atlético-Villarreal          (A-B, 1.30 M)
│ Domingo   16h  Athletic-Mallorca            (B-B, 0.90 M)
│           18h  Barcelona-Las Palmas         (A-C, 1.00 M)
│           20h  Valladolid-Betis             (C-B, 0.75 M)
│           20h  Leganés-Real Madrid          (C-A, 1.00 M)
│ Lunes     20h  Rayo Vallecano-Eibar         (B-C, 0.75 M)
└──────────────────────────────────────────────────────────────────────┘

┌─ Jornada 38 ──────────────────────────────────────── Audiencia: 6.4455 M ─┐
│ Viernes   20h  Eibar-Sevilla                (C-B, 0.75 M)
│ Sábado    16h  Alavés-R. Sociedad           (C-B, 0.75 M)
│           18h  Mallorca-Valencia            (B-B, 0.90 M)
│           20h  Celta-Barcelona              (B-A, 1.30 M)
│           20h  Athletic-Villarreal          (B-B, 0.90 M)
│ Domingo   16h  Espanyol-Rayo Vallecano      (B-B, 0.90 M)
│           18h  Valladolid-Real Madrid       (C-A, 1.00 M)
│           20h  Osasuna-Getafe               (B-C, 0.75 M)
│           20h  Betis-Atlético               (B-A, 1.30 M)
│ Lunes     20h  Leganés-Las Palmas           (C-C, 0.47 M)
└──────────────────────────────────────────────────────────────────────┘
```

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Referencias

P1 - Principio multiplicativo (posibilidades sin restricciones):

1. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 1 - Complejidad computacional.
2. VIU-03MIAR, Sesión 02, VC1 - Introducción a los Algoritmos (conteo de posibilidades, fuerza bruta).

P2 - Principio de inclusión-exclusión (posibilidades con restricciones):

3. Wikipedia: *Principio de inclusión-exclusión*. https://es.wikipedia.org/wiki/Principio_de_inclusión-exclusión

P3 - Estructura de datos para el espacio de soluciones:

4. VIU-03MIAR, AG1: Problema de las N-Reinas. Representación vectorial: posición = entidad, valor = asignación. Precedente directo de la estructura elegida para el problema de La Liga.

P4 - Función objetivo y algoritmo de fuerza bruta:

5. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 1, Definición de función objetivo y optimización (maximización/minimización).
6. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 4, sec. 4.1, Problema del agente viajero: fuerza bruta como revisión exhaustiva de todas las soluciones factibles.
7. VIU-03MIAR, Sesión 02, VC1 - Introducción a los Algoritmos (enumeración exhaustiva, fuerza bruta).

P5 - Algoritmo voraz (greedy):

8. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 3, sec. 3.1, Algoritmos voraces: características, esquema general y ejemplos de aplicación.

P6 - Líneas futuras (algoritmos alternativos):

9. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 3.2, Vuelta atrás (backtracking).
10. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 3.4, Programación dinámica.
11. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 4.2, Problema de la mochila.
12. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 5.3, Ramificación y poda.
13. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 7, Métodos heurísticos y metaheurísticos (búsqueda tabú, recocido simulado, PSO, GRASP).
14. Reyero Díez, R. *"Algoritmos de Optimización"*, Manual de la asignatura, VIU. Cap. 8, Algoritmos evolutivos y genéticos (colonia de hormigas).

Describe brevemente las lineas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

El problema de asignación de horarios de La Liga admite múltiples extensiones, tanto en los algoritmos empleados como en la formulación del propio problema.

1. Algoritmos más sofisticados

- **Backtracking con poda** (Cap. 3.2): explorar el árbol de asignaciones partido→franja, pero podar ramas que ya no puedan superar la mejor solución encontrada hasta el momento. Mantiene la garantía de optimalidad del enfoque por fuerza bruta, reduciendo drásticamente el espacio explorado.
- **Ramificación y poda** (Cap. 5.3): añadir cotas superiores (por ejemplo, asumiendo que los partidos restantes se asignan a su mejor franja posible) permite podar de forma aún más agresiva.
- **Programación dinámica** (Cap. 3.4): si se discretiza la audiencia, el problema puede formularse como una asignación óptima con subestructura óptima, donde las decisiones parciales se reutilizan para construir la solución global.
- **Metaheurísticas** (Cap. 7): recocido simulado (Cap. 7.2) o búsqueda tabú (Cap. 7.1) pueden encontrar soluciones de alta calidad en tiempos razonables para instancias grandes donde los métodos exactos no escalan.
- **PSO, Optimización por enjambre de partículas** (Cap. 7.3): alternativa a tabú/recocido para la exploración del espacio de soluciones, inspirada en el comportamiento colectivo de bandadas.
- **GRASP** (Cap. 7.4): combina una fase de construcción voraz aleatorizada con una fase de búsqueda local, extensión natural del algoritmo greedy implementado en este trabajo.
- **Algoritmos genéticos** (Cap. 8.2): una población de asignaciones que evoluciona mediante cruces y mutaciones resulta natural para este tipo de problema combinatorio.
- **Colonia de hormigas** (Cap. 8.3): metaheurística bioinspirada donde agentes depositan "feromonas" sobre asignaciones prometedoras, reforzando iterativamente las mejores soluciones.

2. Variaciones del problema

- **Analogía con el problema de la mochila** (Cap. 4.2): el problema de La Liga puede verse como una variante del problema de la mochila, donde los partidos son los objetos, las franjas horarias representan la capacidad y la audiencia actúa como el valor a maximizar. Esta perspectiva permite aplicar técnicas clásicas de optimización combinatoria.
- **Más restricciones**: límites de partidos por día, equipos que no pueden jugar el mismo día por compartición de estadio o ciudad, derbis obligatoriamente en horario estelar.
- **Optimización multiobjetivo**: maximizar la audiencia total y minimizar la desigualdad de audiencia entre equipos (que ningún equipo juegue siempre en la peor franja).
- **Franjas horarias dinámicas**: que el número de franjas varíe según la jornada (por ejemplo, más slots en jornadas de navidad o de mitad de semana).

3. Escalabilidad (aumento de tamaño)

- **Más equipos**: Segunda División cuenta con 22 equipos -> 11 partidos por jornada, o ligas combinadas internacionales.
- **Más franjas horarias**: Nuevos slots de emisión.
- **Ventaja del greedy**: nuestro algoritmo voraz escala en $O(n \cdot k)$ (partidos × franjas), lo que lo hace viable incluso para instancias mucho mayores, aunque a costa de perder la garantía de optimalidad que ofrecen los métodos exactos.